

StreamFlow Phase 1

Containerized Stream Processing Platform

Mason Muscarello Paul Yoo Aleksandr Likhoshva Taranjit Singh

July 15, 2026

Revature

- 01. Architectural Objectives
- 02. System Architecture & Data Flow
- 03. The Data Contract & Quality Framework
- 04. Processing & Orchestration Strategy
- 05. End-to-End Execution & Metrics
- 06. Engineering Challenges & Key Takeaways

Project Objectives & Scope

Core Objectives

- **Containerized Flow:**
Standardize the developer environment locally using Docker.
- **In-Stream Auditing:**
Validate schemas and enforce strict constraints at ingestion.

Architecture Shift

- **Decoupled Design:** Isolate continuous event streams from scheduled downstream aggregate runs.
- **Bounded Batches:**
Transition from infinite stream loops to repeatable daily batch summaries.

Data Processing Pipeline

1. **Event Ingestion:** Python Event Producer publishes transactional events to a local Apache Kafka broker topic.
2. **Streaming Ingestion:** Spark Structured Streaming consumes events incrementally, parses payloads, and checks quality.
3. **Isolation Storage:** Valid events are saved to `data/raw/`; rejected events go to `data/rejects/`.
4. **Workflow Orchestration:** Apache Airflow coordinates batch jobs, executing daily summaries and validating downstream files.

Event Data Contract & Domain

Retail Inventory Management Domain

Using transactional inventory events (`inventory.stock_change`) generated by the streamflow producer.

Envelope Schema

- `event_id`: String (UUID)
- `event_type`: String
- `event_ts`: ISO 8601 Timestamp
- `source`: Identifier
- `payload`: Nested struct

Payload Details

- `sku_id`: Product ID
- `quantity_delta`: Int
- `current_stock_level`: Int
- `checkout_lane`: Nullable String
- `price`: Double

Stream Ingestion & Checkpointing

Spark Structured Streaming (`streaming_ingest.py`)

- Consumes micro-batches incrementally from the Apache Kafka broker.
- Integrates externalized configs (bootstrap servers, topic filters) loaded from `pipeline.yml` or environment variables.
- Outputs processed records directly to high-performance Parquet format.

Fault Tolerance & Durability

- Utilizes Spark DFS checkpointing (`data/checkpoints/raw`) to track read progress.
- Provides state recovery to ensure exactly-once ingestion semantics upon container restart.

Data Quality & Rules Engine

Validation Framework (`quality.py`)

Performs checks at the processing edge before storage:

- **Structure:** Non-empty fields, valid JSON parse.
- **Constraints:** Match allowed sources/event types, valid ISO timestamps.
- **Deduplication:** Identifies duplicates in active micro-batches and joins against historical raw logs to prevent double-processing.

Valid Stream (Parquet)

Clean, schema-compliant events written directly to `data/raw/`.

Rejected Stream (Audit)

Failing records written to `data/rejects/` with appended failure reasons.

Workflow Orchestration

DAG: `streamflow_daily_summary`

Airflow coordinates bounded batch aggregation cycles (no infinite streaming loops inside orchestrator).

Step 1: Aggregation Job

SparkSubmitOperator:

- Triggers batch job `daily_summary.py`.
- Summarizes metrics from raw Parquet data.
- Overwrites curated analytical summaries.

Step 2: Output Validation

PythonOperator:

- Validates curated directories exist.
- Checks output files are non-empty.
- Emits job status and metadata.

End-to-End Pipeline Verification

- **Setup:** Spin up environment using `make up`. Check Docker health status.
- **Ingestion:** Start the python-based producer to begin streaming transactional events into Kafka.
- **Aggregation:** Run tests or trigger Airflow DAG from web UI (`localhost:8080`) to aggregate records.
- **Verification:** Inspect output directories (`data/raw/`, `data/rejects/`, `data/curated/`) to verify pipeline success.

Key Architecture Bottlenecks Resolved

- **Docker Networking:** Configured dual-listener setup inside Compose for Kafka (`localhost:9092` for host vs. internal container routing via `kafka:29092`).
- **DataFrame Schema Parsing:** Safely flattened multi-nested JSON structures directly inside PySpark streaming using explicit spark schema structures.
- **DFS Checkpoint Permissions:** Resolved local folder mount read/write access and umask restrictions under Dockerized Spark executions.

Enterprise Scaling Roadmap

- **Object Storage Migration:** Replace local folder mounts with local S3-compatible storage engine (MinIO) for enterprise testing.
- **Data Testing Layer:** Implement Great Expectations directly into validation pipelines to check data health before parquet writes.
- **Advanced Streaming Aggregation:** Introduce sliding time-window aggregations and multi-topic Kafka routing workflows.

Conclusion & Q&A